

TIME TRACKING APP

Especificação Técnica e Plano de Desenvolvimento — MVP

Versão: 1.1 **Plataforma:** Windows Desktop **Tipo:** Aplicação local/offline para controle de tempo de tarefas **Banco:** SQLite local **Desenvolvimento assistido por Claude Code**

NOTA DE REVISÃO PRÉ-DESENVOLVIMENTO (Fase 0 do usuário, antes da Fase 0 do Claude Code)

Esta nota documenta decisões tomadas para resolver ambiguidades identificadas nesta especificação antes do início do desenvolvimento. As seções ao longo do documento já foram atualizadas para refletir essas decisões — esta nota serve apenas de registro e contexto para quem ler o documento depois.

1. **Referência visual (interfaceref.png):** é uma referência de **estilo** apenas — paleta escura, tipografia e acento roxo/lilás. Não define comportamento de layout. O comportamento funcional de sidebar (Seção 19) e painel direito (Seção 20) segue exatamente o texto dessas seções, independentemente do que a imagem mostra (ver nota na Seção 31).
2. **Edição de datas/horários de tarefa (Seções 17-18):** como uma Task pode ter múltiplas TimeEntry, a edição direta de StartedAt/EndedAt só é permitida quando a tarefa possui exatamente uma TimeEntry. Com múltiplas sessões, o painel exibe o tempo total agregado e os horários de forma somente leitura, com indicador visual de "múltiplas sessões". Edição granular por sessão fica para o backlog.
3. **Campo Task.Status:** removido do modelo de dados do MVP por não haver nenhuma ação de UI definida para alterá-lo. Fica registrado no backlog (Seção 62) para reintrodução junto de uma ação concreta de UI.
4. **Exclusão de tarefas:** passa a ser requisito do MVP (não mais opcional), com confirmação obrigatória — sem Status/arquivamento, o usuário precisa de alguma forma de remover tarefas indesejadas.
5. **"Limpar histórico" (Seção 27):** escopo definido explicitamente — remove todas as Task e TimeEntry; as Tag são preservadas.
6. **Versão do .NET:** fixada em .NET 8 (LTS).
7. **Local do banco de dados:** fixado em %LocalAppData%\TimeTracking\timetracking.db.

8. **Diálogos de confirmação:** não utilizar `MessageBox.Show` nativo do Windows; criar um componente de diálogo/modal próprio dentro do design system (Seção 30), consistente com o tema claro/escuro.
 9. **Tema "Sistema" (Seção 26):** detectado via leitura do registro do Windows (`HKCU\Software\Microsoft\Windows\CurrentVersion\Themes\Personalize\AppsUseLightTheme`) ou API equivalente do .NET 8/WPF.
 10. **Fluxo de criação de tarefa (Seção 22):** reaproveita o mesmo painel/drawer de edição (Seção 20), aberto em estado "nova tarefa" (campos vazios), em vez de uma UI separada.
 11. **Validação de nome (Seção 40):** limite máximo definido em 200 caracteres para Task, 100 para Tag.
 12. **Estratégia de testes de persistência (Seção 47):** usar SQLite com `Data Source=:memory:` (conexão aberta mantida viva durante o teste), em vez do provider `InMemory` do EF Core, que não valida integridade relacional.
-

1. OBJETIVO DO PROJETO

O projeto consiste em uma aplicação desktop para Windows destinada ao controle e registro do tempo gasto em tarefas.

O usuário deverá conseguir:

- criar tarefas;
- iniciar o contador de uma tarefa;
- pausar o contador;
- interromper/encerrar o contador;
- visualizar o tempo registrado;
- editar tarefas posteriormente;
- alterar manualmente informações de uma tarefa mesmo após o timer ter sido concluído;
- utilizar tags personalizadas para categorizar tarefas;
- criar, editar e excluir tags;
- alterar a aparência da aplicação entre temas claro e escuro;
- manter todos os dados localmente na máquina do usuário.

A primeira versão deve permanecer deliberadamente simples.

O objetivo do MVP não é criar uma plataforma completa de produtividade, mas sim construir um **time tracker desktop confiável, organizado e extensível**.

2. PRINCÍPIOS FUNDAMENTAIS DO PROJETO

O desenvolvimento deve seguir estas regras:

1. Priorizar simplicidade.
 2. Não implementar funcionalidades não especificadas no MVP.
 3. Não criar infraestrutura desnecessária.
 4. Não criar backend remoto.
 5. Não criar sistema de login.
 6. Não utilizar banco de dados remoto.
 7. Todos os dados devem permanecer localmente.
 8. A arquitetura deve permitir futuras extensões sem exigir uma reescrita completa.
 9. A interface deve ser moderna, limpa e agradável, mas sem sacrificar funcionalidade.
 10. O código deve priorizar legibilidade e manutenção.
 11. Responsabilidades devem ser separadas.
 12. Regras de negócio não devem ficar diretamente nas Views.
 13. O banco não deve ser manipulado diretamente pelas Views ou ViewModels.
 14. O timer deve ser persistente e baseado em timestamps, não apenas em um contador em memória.
 15. Uma única tarefa poderá estar com o timer em execução simultaneamente.
 16. O usuário deverá poder editar dados de tarefas já encerradas.
 17. Toda ação destrutiva deve possuir confirmação.
 18. O aplicativo deve funcionar sem conexão com a internet.
-

3. TECNOLOGIAS

3.1 Stack principal

Utilizar:

- **C#**
- **.NET 8 (LTS)**
- **WPF**
- **XAML**
- **MVVM**
- **SQLite**
- **Entity Framework Core**

3.2 Bibliotecas recomendadas

Utilizar, quando apropriado:

- `CommunityToolkit.Mvvm`
- `Microsoft.Extensions.DependencyInjection`
- `Microsoft.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore.Sqlite`

Serilog pode ser utilizado futuramente ou incluído apenas se houver necessidade real de logging estruturado.

Não adicionar bibliotecas apenas por conveniência quando uma solução simples utilizando .NET/WPF já existir.

4. JUSTIFICATIVA DA ARQUITETURA

A aplicação será executada localmente em máquinas Windows.

Por isso, não será criada uma arquitetura web tradicional com:

Frontend

↓

API

↓

Servidor

↓

Banco remoto

A aplicação utilizará:

WPF

↓

MVVM

↓

Application Services

↓

Repositories / EF Core

↓

SQLite

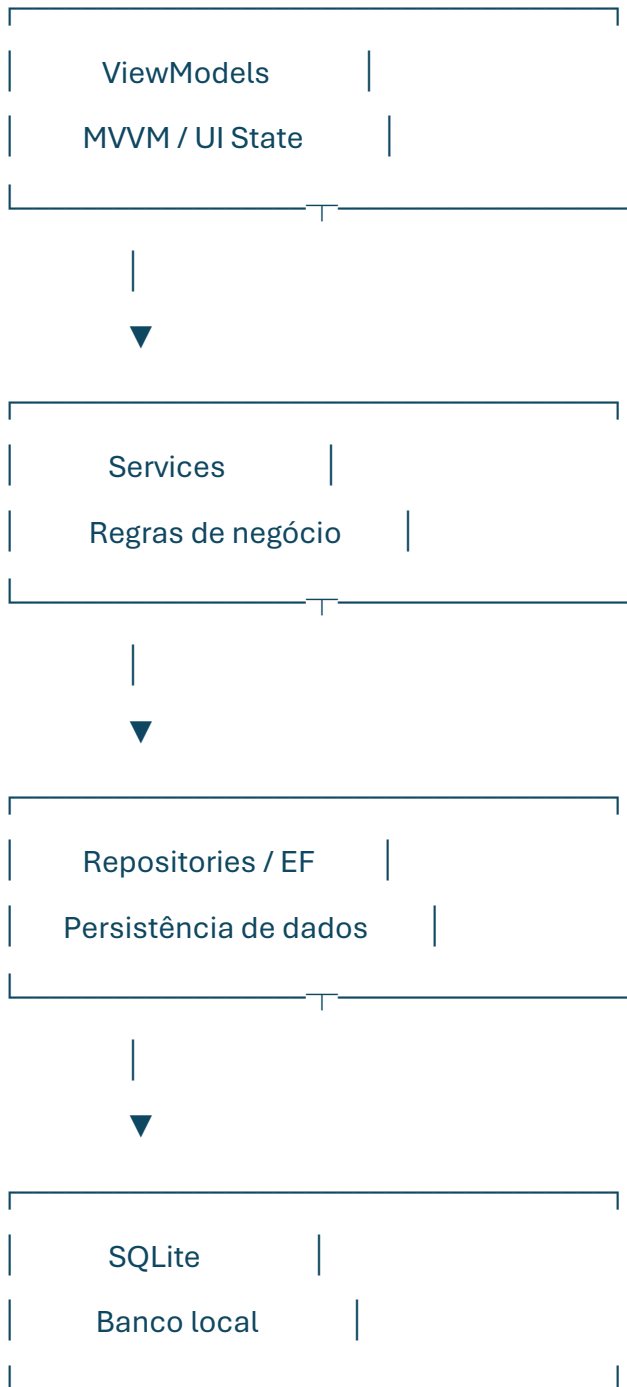
Essa abordagem reduz a complexidade do projeto e atende diretamente ao requisito de execução local.

5. ARQUITETURA

A aplicação deve possuir separação clara entre apresentação, lógica de aplicação e persistência.

Estrutura conceitual:





Regra importante

Views não devem acessar diretamente:

- DbContext;
- SQLite;

- repositories;
- SQL;
- lógica de negócio.

ViewModels também não devem conter lógica complexa de persistência.

6. MODELO DE DOMÍNIO

O núcleo do sistema será composto por três entidades principais:

Tag

|

| 1:N



Task

|

| 1:N



TimeEntry

6.1 Tag

Representa uma categoria personalizada criada pelo usuário.

Campos:

Id

Name

Description

Color

CreatedAt

UpdatedAt

Observações

- Id: identificador interno.
- Name: nome da tag.
- Description: descrição opcional.
- Color: cor utilizada visualmente na interface.
- CreatedAt: data de criação.
- UpdatedAt: última alteração.

Nesta versão NÃO implementar:

- ícone;
- imagem;
- emoji personalizado;
- hierarquia de tags.

Ícones poderão ser adicionados futuramente.

7. TASK

A entidade Task representa aquilo que o usuário está realizando.

Campos:

Id

Name

Description

TagId

CreatedAt

UpdatedAt

Relacionamento:

Task.TagId → Tag.Id

Status da tarefa (fora do MVP)

O campo Status (Active/Completed/Archived), presente em versões anteriores desta spec, foi **retirado do modelo de dados do MVP**.

Motivo: nenhuma fase do MVP define uma ação de UI para alterá-lo, e um campo sem forma de ser modificado pelo usuário não deve existir no banco (Regra 10 — Não overengineer).

Esse campo poderá ser reintroduzido em versão futura, junto de uma ação concreta de UI (ex.: "marcar como concluída" ou "arquivar"). Ver Seção 62 — Backlog Futuro.

8. TIME ENTRY

A entidade TimeEntry representa uma sessão efetiva de trabalho.

Campos:

Id

TaskId

StartedAt

EndedAt

DurationSeconds

Relacionamento:

TimeEntry.TaskId → Task.Id

Motivo da separação

Uma tarefa pode possuir várias sessões.

Exemplo:

Tarefa:

Desenvolver API

Sessão 1:

10:00 → 11:00

Sessão 2:

14:00 → 15:30

Tempo total:

2h30

Não representar isso simplesmente como:

10:00 → 15:30

pois isso contabilizaria incorretamente períodos em que o usuário não estava trabalhando.

9. REGRAS DO BANCO DE DADOS

O banco deve ser SQLite.

O arquivo do banco deve ser armazenado em um local apropriado para dados locais da aplicação, e não depender do diretório de instalação.

Caminho definido: %LocalAppData%\TimeTracking\timetracking.db.

O aplicativo deve criar/inicializar o banco automaticamente quando necessário.

Utilizar Entity Framework Core migrations.

Integridade

Uma TimeEntry deve sempre pertencer a uma Task.

Uma Task pode possuir uma Tag, mas a associação deve poder ser nula caso o usuário não queira categorizar a tarefa.

Não permitir que a exclusão de uma Tag apague automaticamente as Tasks associadas.

Ao excluir uma Tag que esteja sendo utilizada:

Tag → removida

Task → permanece

Task.TagId → null

Solicitar confirmação antes da exclusão.

Ao excluir uma Task (Seção 23), suas TimeEntry associadas são excluídas junto (cascade delete) — nesse sentido a relação Task → TimeEntry é diferente da relação Tag → Task.

10. TIMER

O timer é a funcionalidade central da aplicação.

Estados possíveis:

Stopped

Running

Paused

O estado do timer não deve ser tratado simplesmente como um número incrementado a cada segundo.

O tempo deve ser calculado com base nos timestamps persistidos.

11. COMPORTAMENTO DO PLAY

Quando o usuário clicar em:

► Play

deve ser criada uma nova TimeEntry.

Exemplo:

StartedAt = 2026-08-29 15:30:00

EndedAt = null

Enquanto a sessão estiver ativa:

Task Timer State = Running

O tempo exibido na interface deve ser calculado utilizando:

CurrentTime - StartedAt

ou equivalente.

Não depender de um contador em memória para determinar o tempo real.

12. COMPORTAMENTO DO PAUSE

Ao clicar em:

II Pause

a sessão atual deverá ser encerrada:

EndedAt = CurrentTime

A tarefa deixa de possuir uma sessão ativa.

O tempo total da tarefa deverá considerar todas as sessões existentes.

Exemplo:

Session 1

10:00 → 11:00

Session 2

14:00 → 14:30

Total:

1h30

13. COMPORTAMENTO DO PLAY APÓS PAUSE

Se uma tarefa pausada for iniciada novamente:

► Play

não modificar a sessão anterior.

Criar uma nova TimeEntry.

Exemplo:

TimeEntry 1

10:00 → 11:00

TimeEntry 2

14:00 → null

14. COMPORTAMENTO DO STOP

O botão:

■ Stop

deve encerrar a sessão atual.

Se existir uma TimeEntry ativa:

EndedAt = CurrentTime

A tarefa deverá deixar de possuir timer ativo.

Não assumir que parar o timer significa necessariamente excluir a tarefa.

15. APENAS UMA TAREFA ATIVA

Somente uma tarefa poderá possuir uma sessão de timer em execução.

Se o usuário tentar iniciar outra tarefa enquanto existe uma tarefa em execução:

A tarefa "Tarefa A" está em execução.

Deseja pausá-la e iniciar "Tarefa B"?

Opções:

Cancelar

Iniciar

Caso o usuário confirme:

Tarefa A

Running → Paused

Tarefa B

Stopped → Running

Não permitir dois timers simultaneamente no MVP.

Este diálogo deve usar o componente de confirmação do design system (Seção 30), não MessageBox.Show nativo.

16. PERSISTÊNCIA DO TIMER

O timer deve continuar consistente mesmo se o aplicativo for fechado e aberto novamente.

Exemplo:

15:00

Usuário inicia tarefa

15:20

Aplicativo é fechado

15:40

Aplicativo é aberto

Ao abrir novamente, se existir uma `TimeEntry` sem `EndedAt`, o aplicativo deverá reconhecer que existe uma sessão ativa e reconstruir o tempo utilizando o timestamp original.

O tempo não deve depender do fato de o processo da aplicação ter permanecido aberto.

17. EDIÇÃO DE TAREFAS

Tarefas poderão ser editadas mesmo após o timer ter sido encerrado.

O usuário poderá editar sempre:

- nome;
- descrição;
- tag.

Além disso, o usuário poderá editar data/horário de início e término, com a seguinte regra:

- **quando a tarefa possuir exatamente uma `TimeEntry`:** os campos de data/hora editam essa sessão diretamente.

- **quando a tarefa possuir mais de uma TimeEntry:** os campos exibem o início da primeira sessão e o término da última em modo **somente leitura**, junto com um indicador visual de "múltiplas sessões" e o tempo total agregado. Edição granular por sessão fica para versão futura (Seção 62).

As alterações deverão ser persistidas no banco.

O usuário deverá clicar em:

Salvar

para confirmar alterações.

Se o painel for fechado sem salvar, alterações não persistidas não devem ser aplicadas.

18. EDIÇÃO DE TIME ENTRIES

A regra de edição de datas/horários está definida na Seção 17: edição direta é permitida apenas quando a tarefa possui uma única TimeEntry; com múltiplas sessões, a exibição é agregada e somente leitura no MVP.

A arquitetura deve permitir futuramente uma edição granular das sessões (lista de TimeEntry editável individualmente).

No MVP, a interface apresenta o início/término da tarefa de maneira simples (Seção 17), mas a estrutura de dados deve preservar as sessões individuais.

Não destruir a arquitetura de TimeEntry apenas para simplificar a interface inicial.

19. SIDEBAR ESQUERDA

A aplicação terá um menu lateral esquerdo inicialmente fechado.

Quando o usuário clicar no ícone hamburger:



o menu deverá abrir.

Itens:

Time Tracking

Tags

Pomodoro

Settings

O usuário poderá fechar o menu:

1. clicando novamente no botão hamburger;
2. clicando fora da área do menu.

O menu deverá possuir animação discreta.

Não exagerar nas animações.

Este comportamento (fechado por padrão, abre como overlay, fecha ao clicar fora) é o comportamento funcional definitivo — ver nota sobre a imagem de referência na Seção 31.

20. SIDEBAR DIREITA / PAINEL DE EDIÇÃO

O painel lateral direito não deverá permanecer permanentemente visível.

Ele deverá aparecer quando o usuário selecionar uma tarefa existente, ou quando clicar em "+ Nova tarefa" (Seção 22) — nesse caso, o mesmo painel abre em estado "nova tarefa", com campos vazios.

O painel deverá permitir:

Nome

Descrição

Tag

Data de início

Hora de início

Data de término

Hora de término

Os campos de data/horário seguem a regra da Seção 17 (editáveis diretamente apenas quando a tarefa tem uma única TimeEntry; somente leitura e agregados quando há múltiplas sessões). No fluxo de criação de tarefa, esses campos ficam ocultos ou desabilitados, já que a tarefa ainda não possui nenhuma TimeEntry.

E:

[Salvar]

O painel poderá ser fechado clicando fora de sua área.

A implementação pode utilizar um Drawer/Overlay visual dentro da janela principal.

Não abrir uma nova janela independente para cada edição se isso prejudicar a experiência de uso.

Este comportamento (drawer/overlay que aparece sob demanda e fecha ao clicar fora) é o comportamento funcional definitivo — ver nota sobre a imagem de referência na Seção 31.

21. TELA TIME TRACKING

Essa será a tela principal.

Deve permitir:

- visualizar tarefas;
- criar nova tarefa;
- iniciar timer;
- pausar timer;
- parar timer;
- selecionar tarefa;
- editar tarefa;
- visualizar tempo registrado.

Estrutura visual sugerida:





O design final pode diferir desse wireframe, desde que respeite as regras funcionais.

22. CRIAÇÃO DE TAREFA

O usuário deverá possuir uma ação claramente identificável:

+ Nova tarefa

Campos mínimos:

Nome

Descrição

Tag

A UI de criação reaproveita o painel direito de edição (Seção 20) em estado "nova tarefa", em vez de uma tela ou modal separado — mantém consistência visual e evita duplicar componentes.

O sistema deverá criar a tarefa sem iniciar automaticamente o timer.

Preferência para o MVP:

Criar tarefa

↓

Tarefa criada

↓

Timer parado

↓

Usuário decide quando iniciar

23. EXCLUSÃO DE TAREFA

Decisão pós-revisão: a exclusão de tarefas **passa a ser um requisito do MVP** (deixou de ser opcional).

Motivo: como o campo Status/arquivamento foi removido do modelo do MVP (Seção 7), sem exclusão o usuário não teria nenhuma forma de remover uma tarefa indesejada da lista, o que compromete o objetivo de organização do produto (Seção 1).

Regras:

- exigir confirmação antes de excluir, usando o componente de diálogo do design system (Seção 30);
 - ao excluir uma Task, suas TimeEntry associadas são excluídas junto (cascade delete) — ver Seção 9;
 - não permitir exclusão acidental (ação separada dos botões de timer);
 - não permitir excluir uma tarefa que possua timer em execução sem antes parar o timer (ou parar automaticamente como parte da exclusão, com aviso claro no diálogo de confirmação).
-

24. TAGS

A tela Tags deverá permitir:

Criar

Editar

Excluir

Cada Tag terá:

Nome

Descrição

Cor

Não implementar ícones nesta versão.

A cor deverá ser utilizada visualmente nas tarefas, chips, badges ou indicadores.

Exemplo:

● Desenvolvimento

● Estudos

● Trabalho

25. TELA DE TAGS

Estrutura sugerida:

Tags

● Desenvolvimento	
Projetos de programação	
Editar Excluir	

● Estudos	
Faculdade	
Editar Excluir	

+ Nova tag

A interface deve priorizar leitura rápida.

26. SETTINGS

Settings deverá permanecer pequeno no MVP.

Seções:

Aparência

Tema

☐ Dark

☐ Light

☐ Sistema

Detecção do tema "Sistema": via leitura do registro do Windows (HKCU\Software\Microsoft\Windows\CurrentVersion\Themes\Personalize\AppsUseLightTheme) ou API equivalente disponível no .NET 8/WPF.

Dados

[Limpar histórico]

Sobre

Time Tracking

Versão X.X.X

Não adicionar dezenas de configurações sem necessidade.

27. LIMPAR HISTÓRICO

A ação deverá ser considerada destrutiva.

Escopo definido: remove todas as Task e todas as TimeEntry do banco. As Tag cadastradas são preservadas (permanecem disponíveis para uso em tarefas futuras).

Ao clicar:

Limpar histórico

mostrar confirmação (usando o componente de diálogo do design system, Seção 30):

Excluir histórico?

Esta ação removerá todas as tarefas e registros de tempo. As tags cadastradas serão preservadas.

Essa ação não pode ser desfeita.

[Cancelar] [Excluir]

Nunca executar essa ação imediatamente.

28. DARK MODE

O Dark Mode deve possuir aparência confortável e profissional.

Evitar:

- preto absoluto em toda a interface;
- excesso de cores saturadas;
- sombras exageradas;
- muitos gradientes.

Paleta inicial sugerida:

Background:

#181614

Surface:

#211F1C

Surface Elevated:

#292622

Border:

#3A3631

Primary:

#C89B6D

Primary Hover:

#D8AE82

Text Primary:

#F2ECE5

Text Secondary:

#B8AEA3

Text Muted:

#81786F

Success:

#7FAE82

Warning:

#D3A85C

Danger:

#C87575

Essas cores são uma proposta inicial e poderão ser ajustadas durante a fase de polish.

29. LIGHT MODE

Sugestão de paleta:

Background:

#F7F3EE

Surface:

#FFFDF9

Surface Elevated:

#FFFFFF

Border:

#DED6CC

Primary:

#9A6F45

Primary Hover:

#825B37

Text Primary:

#2C2926

Text Secondary:

#665F58

Text Muted:

#8C837A

Success:

#628A66

Warning:

#A47D3C

Danger:

#A95F5F

O Light Mode deve manter contraste adequado e não parecer simplesmente uma inversão do Dark Mode.

30. DESIGN SYSTEM

O projeto deve possuir um sistema visual centralizado.

Não espalhar valores de cores, espaçamentos e estilos diretamente por dezenas de Views.

Criar recursos reutilizáveis para:

- cores;
- tipografia;

- espaçamento;
- bordas;
- botões;
- inputs;
- cards;
- badges;
- ícones;
- diálogos de confirmação;
- estados de hover;
- estados de foco;
- estados desabilitados.

Diálogos de confirmação (Seções 15, 23, 27) devem usar um componente próprio do design system, **não** `MessageBox.Show` nativo do Windows, para manter consistência visual com o tema claro/escuro.

Exemplo conceitual:

Resources/

└─ Themes/

| └─ Dark.xaml

| └─ Light.xaml

|

└─ Styles/

| └─ Buttons.xaml

| └─ Inputs.xaml

| └─ Cards.xaml

| └─ Dialogs.xaml

| └─ Navigation.xaml

|

31. DIRETRIZES VISUAIS

A interface deve transmitir:

- simplicidade;
- organização;
- produtividade;
- conforto visual;
- modernidade.

Evitar aparência excessivamente corporativa.

Também evitar transformar o aplicativo em uma interface excessivamente decorativa.

O usuário deve conseguir olhar para a tela e entender imediatamente:

Qual tarefa está ativa?

Quanto tempo ela está levando?

Quais tarefas existem?

Como iniciar/parar o timer?

Essas informações possuem prioridade visual.

Nota sobre a referência visual (interfaceref.png)

A imagem interfaceref.png anexada ao projeto é uma referência de **estilo** apenas: paleta escura, tipografia e acento roxo/lilás.

Ela **não** define comportamento de layout. O comportamento funcional de sidebar (Seção 19) e painel direito (Seção 20) — abertura/fechamento, overlay vs. coluna fixa, fechamento ao clicar fora — segue exatamente o texto dessas seções, independentemente do que a imagem mostra.

32. ESTRUTURA DE PASTAS

Estrutura inicial recomendada:

TimeTracking/

```
|
|
|└─ App.xaml
|└─ App.xaml.cs
|└─ TimeTracking.csproj
|
|└─ Models/
|  |└─ Task.cs
|  |└─ Tag.cs
|  └─ TimeEntry.cs
|
|└─ Data/
|  |└─ AppDbContext.cs
|  └─ Migrations/
|
|└─ Repositories/
|  |└─ ITaskRepository.cs
|  |└─ TaskRepository.cs
|  |└─ ITagRepository.cs
|  |└─ TagRepository.cs
|  |└─ ITimeEntryRepository.cs
|  └─ TimeEntryRepository.cs
|
|└─ Services/
|  |└─ TaskService.cs
|  |└─ TagService.cs
|  └─ TimerService.cs
```

```
|   ├── NavigationService.cs
|   └── ThemeService.cs
|
|   ├── ViewModels/
|   |   ├── MainViewModel.cs
|   |   ├── TimeTrackingViewModel.cs
|   |   ├── TaskEditorViewModel.cs
|   |   ├── TagsViewModel.cs
|   |   ├── TagEditorViewModel.cs
|   |   └── SettingsViewModel.cs
|   |
|   ├── Views/
|   |   ├── MainWindow.xaml
|   |   ├── TimeTrackingView.xaml
|   |   ├── TagsView.xaml
|   |   ├── SettingsView.xaml
|   |   |
|   |   └── Components/
|   |       ├── Sidebar.xaml
|   |       ├── TaskCard.xaml
|   |       ├── TaskEditorPanel.xaml
|   |       └── TagChip.xaml
|   |
|   ├── Resources/
|   |   ├── Themes/
|   |   |   └── Dark.xaml
```

| | └─ Light.xaml

| |

| └─ Styles/

| └─ Icons/

|

└─ Helpers/

Essa estrutura pode ser adaptada se o projeto demonstrar necessidade real.

Não criar abstrações vazias apenas para preencher pastas.

33. RESPONSABILIDADES

Models

Representam entidades e dados do domínio.

Não colocar lógica de interface.

ViewModels

Controlam:

- estado da UI;
- comandos;
- propriedades observáveis;
- comunicação com Services.

Services

Contêm regras de negócio.

Exemplos:

TimerService

TaskService

TagService

ThemeService

Repositories

Responsáveis pela persistência.

Views

Responsáveis pela apresentação.

34. TIMER SERVICE

O TimerService deve centralizar as regras do timer.

Responsabilidades:

- iniciar timer;
- pausar timer;
- parar timer;
- verificar timer ativo;
- recuperar timer ativo ao iniciar aplicação;
- calcular tempo decorrido;
- impedir múltiplos timers simultâneos;
- persistir mudanças.

A ViewModel não deve implementar diretamente a lógica de cálculo de tempo.

35. NAVEGAÇÃO

A aplicação deverá possuir uma janela principal.

A área de conteúdo deverá trocar a View conforme a opção escolhida na Sidebar.

Rotas lógicas:

TimeTracking

Tags

Pomodoro

Settings

Pomodoro deverá existir na navegação somente se isso fizer parte do escopo visual definido, mas sua funcionalidade não deverá ser implementada no MVP.

Pode ser apresentado como:

Pomodoro

Em breve

ou permanecer temporariamente indisponível.

36. POMODORO — FUTURO

Pomodoro NÃO faz parte do núcleo do MVP.

Preparar apenas a arquitetura de navegação para sua futura inclusão.

Não implementar:

- timer Pomodoro;
- ciclos;
- pausas;
- notificações;
- estatísticas de Pomodoro.

Esses itens pertencem a uma fase futura.

37. DASHBOARD — FUTURO

Uma futura versão poderá possuir Dashboard.

Possíveis funcionalidades:

Tempo total da última semana

Tempo por tarefa

Tempo por tag

Tarefa com maior atividade

Categoria com maior atividade

Exemplo:

Últimos 7 dias

Desenvolvimento 

Estudos 

Trabalho 

Porém:

NÃO implementar no MVP.

A estrutura atual de TimeEntry e Tag deve apenas permitir que isso seja implementado posteriormente.

38. FUNCIONALIDADES FORA DO MVP

Não implementar neste momento:

- login;
- contas;
- sincronização;
- cloud;
- API;
- servidor;
- banco remoto;
- colaboração;
- dashboard;
- gráficos avançados;
- relatórios;
- exportação;
- backup em nuvem;
- subtarefas;
- projetos;

- metas;
- notificações complexas;
- ícones de tags;
- integrações externas.

Essas funcionalidades podem ser registradas como backlog futuro.

39. TRATAMENTO DE ERROS

Erros devem ser tratados de maneira amigável.

Nunca exibir exceções técnicas diretamente ao usuário.

Exemplo ruim:

SQLException: SQLite Error 19...

Exemplo esperado:

Não foi possível salvar a tarefa.

Verifique os dados e tente novamente.

Detalhes técnicos devem permanecer em logs quando apropriado.

40. VALIDAÇÃO

Task

Nome:

- obrigatório;
- não pode ser vazio;
- máximo de 200 caracteres.

Descrição:

- opcional.

Tag:

- opcional.

Tag

Nome:

- obrigatório;
- não pode ser vazio;
- máximo de 100 caracteres.

Cor:

- deve possuir valor válido.
-

41. ACESSIBILIDADE E UX

A interface deverá:

- possuir foco visual claro;
- permitir navegação lógica;
- possuir botões identificáveis;
- evitar textos excessivamente pequenos;
- utilizar contraste adequado;
- diferenciar estados por mais do que apenas cor quando necessário.

Botões de timer devem possuir ícone e/ou texto suficientemente claro.

42. RESPONSIVIDADE DA JANELA

Embora seja uma aplicação desktop, a interface não deve depender de uma única resolução.

A janela deverá funcionar adequadamente em diferentes tamanhos.

Não utilizar posições absolutas para estruturar toda a interface.

Priorizar:

- Grid;
- StackPanel;

- DockPanel;
 - layouts responsivos do WPF;
 - dimensões mínimas razoáveis.
-

43. DESEMPENHO

O aplicativo deve ser leve.

Não executar consultas ao banco a cada atualização visual do contador.

O timer visual pode atualizar a cada segundo, mas isso não significa executar uma operação de escrita no SQLite a cada segundo.

O banco deve registrar eventos significativos:

Play

Pause

Stop

Edit

Create

Delete

O contador exibido na tela deve ser calculado em memória a partir dos timestamps persistidos.

44. SEGURANÇA E PRIVACIDADE

Como o aplicativo é local:

- não enviar dados para servidores;
- não criar telemetria externa;
- não enviar tarefas para serviços de terceiros;
- não exigir internet;
- não coletar informações desnecessárias.

O banco local deve ser tratado como dado do usuário.

45. VERSIONAMENTO DO BANCO

Utilizar migrations do Entity Framework Core.

Não modificar silenciosamente o banco existente de maneira incompatível.

Alterações futuras de estrutura devem ser representadas por migrations.

46. LOGGING

Caso seja implementado logging:

Registrar informações úteis para diagnóstico, como:

- inicialização da aplicação;
- erros de banco;
- erros de persistência;
- falhas inesperadas.

Não registrar dados desnecessários ou conteúdo sensível.

47. TESTES

As regras críticas deverão possuir testes.

Prioridade:

Timer

Play

Pause

Play

Stop

Tempo

10:00 → 11:30

= 90 minutos

Sessões

10:00 → 11:00

14:00 → 15:00

Total = 2 horas

Persistência

Iniciar timer

Fechar aplicação

Abrir aplicação

Verificar timer

Tags

Criar

Editar

Excluir

Associar

Observação técnica

Testes de persistência devem usar SQLite com Data Source=:memory: (conexão aberta mantida viva durante o teste), **não** o provider InMemory do EF Core — esse provider não valida integridade relacional (chaves estrangeiras, constraints), o que mascararia bugs reais de persistência.

48. DESENVOLVIMENTO POR FASES

Esta é uma regra fundamental do projeto.

Claude Code deve executar somente UMA fase por vez.

Após concluir uma fase:

1. parar;
2. informar o que foi implementado;
3. informar os arquivos alterados;

4. informar testes executados;
5. informar eventuais problemas;
6. aguardar aprovação explícita do usuário.

Claude Code NÃO deve automaticamente iniciar a próxima fase.

49. FASE 0 — ANÁLISE E PLANEJAMENTO

Objetivo:

Validar a arquitetura antes de escrever código.

Claude deve:

- analisar esta especificação;
- verificar possíveis inconsistências;
- propor ajustes técnicos somente quando necessários;
- apresentar a arquitetura final;
- apresentar o modelo de banco;
- apresentar o fluxo do timer;
- apresentar a estrutura de pastas;
- identificar dúvidas ou decisões que precisam ser tomadas.

Claude NÃO deve:

- criar código da aplicação;
- criar todas as telas;
- implementar funcionalidades.

Critério de aceite

A arquitetura deve estar definida e aprovada pelo usuário.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

50. FASE 1 — CRIAÇÃO DO PROJETO

Objetivo:

Criar a fundação do projeto.

Implementar:

- projeto WPF;
- .NET;
- dependências;
- MVVM;
- DI;
- estrutura de pastas;
- configuração inicial.

Ainda NÃO implementar:

- timer;
- CRUD completo;
- telas completas;
- dashboard;
- Pomodoro.

Critério de aceite

A aplicação deve:

- compilar;
- executar;
- abrir a janela principal;
- não apresentar erros de inicialização.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

51. FASE 2 — BANCO DE DADOS

Implementar:

- DbContext;
- Task;
- Tag;
- TimeEntry;
- relacionamentos;
- migrations;
- criação do banco;
- configuração de armazenamento local.

Critério de aceite

O aplicativo deve conseguir:

- criar banco;
- aplicar migrations;
- conectar ao SQLite;
- executar operações básicas de persistência.

Ainda não implementar interface completa.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

52. FASE 3 — SHELL E NAVEGAÇÃO

Implementar:

- MainWindow;
- Sidebar esquerda;
- botão hamburger;
- abertura/fechamento;

- fechamento ao clicar fora;
- navegação;
- área de conteúdo.

Criar placeholders:

Time Tracking

Tags

Pomodoro

Settings

Critério de aceite

O usuário deve conseguir:

abrir menu

↓

selecionar página

↓

visualizar página

↓

fechar menu

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

53. FASE 4 — CRUD DE TASKS

Implementar:

- listar tarefas;
- criar tarefa;
- editar tarefa;
- excluir tarefa (com confirmação);

- persistir alterações;
- validação.

Ainda não implementar o timer completo.

Critério de aceite

O usuário deve conseguir:

Criar tarefa

↓

Visualizar tarefa

↓

Selecionar tarefa

↓

Editar

↓

Salvar

↓

Visualizar alteração

e:

Selecionar tarefa

↓

Excluir

↓

Confirmar

↓

Tarefa removida da lista

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

54. FASE 5 — TIMER

Implementar:

- Play;
- Pause;
- Stop;
- TimeEntry;
- TimerService;
- cálculo por timestamps;
- uma única tarefa ativa;
- recuperação após reinicialização.

Esta é uma fase crítica.

Testar especialmente:

Play

↓

Pause

↓

Play

↓

Stop

e:

Play

↓

Fechar aplicação

↓

Abrir aplicação

Critério de aceite

O timer deve permanecer matematicamente correto.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

55. FASE 6 — PAINEL DE EDIÇÃO

Implementar o painel lateral direito.

Deve permitir:

- editar nome;
- editar descrição;
- editar tag;
- editar início/término (regra da Seção 17: direto quando há uma única TimeEntry; somente leitura agregado quando há múltiplas);
- salvar;
- cancelar;
- fechar clicando fora.

Critério de aceite

Uma tarefa já encerrada deve poder ser editada sem perder seu histórico de tempo.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

56. FASE 7 — TAGS

Implementar:

- lista de tags;
- criar;
- editar;
- excluir;

- cor;
- integração com tarefas.

Testar comportamento de exclusão de tag utilizada por tarefas.

Critério de aceite

Excluir uma tag não deve excluir as tarefas associadas.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

57. FASE 8 — SETTINGS E TEMAS

Implementar:

- Dark Mode;
- Light Mode;
- tema "Sistema" (detecção via registro do Windows);
- preferência de tema;
- limpar histórico (escopo: Task + TimeEntry, preservando Tag);
- confirmação de exclusão;
- tela About.

Critério de aceite

O usuário deve conseguir alternar o tema sem reiniciar a aplicação, caso a implementação permita isso de maneira limpa.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

58. FASE 9 — POLISH DE UI/UX

Somente agora realizar refinamento visual.

Avaliar:

- espaçamentos;

- tipografia;
- cores;
- estados de hover;
- animações;
- cards;
- sidebar;
- painel direito;
- feedback visual;
- mensagens;
- ícones.

Não adicionar funcionalidades novas.

Objetivo:

melhorar a experiência das funcionalidades já existentes.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

59. FASE 10 — TESTES E ESTABILIZAÇÃO

Executar testes completos.

Testar:

- criação;
- edição;
- exclusão;
- tags;
- timer;
- pausa;
- retomada;

- stop;
- reinicialização;
- banco;
- temas;
- histórico;
- erros;
- estados vazios;
- entradas inválidas.

Corrigir bugs encontrados.

Não adicionar funcionalidades novas.

Ao terminar:

PARAR E AGUARDAR APROVAÇÃO.

60. FASE 11 — BUILD E EMPACOTAMENTO

Somente após o MVP estar aprovado.

Preparar:

- Release build;
- publicação para Windows;
- banco local;
- configuração de armazenamento;
- instruções de execução.

Avaliar posteriormente opções como:

Self-contained

Framework-dependent

Installer

Não adicionar atualizador automático neste momento.

61. DEFINITION OF DONE DO MVP

O MVP só será considerado concluído quando:

- ☐ aplicação inicia no Windows;
- ☐ banco SQLite funciona localmente;
- ☐ usuário pode criar tarefas;
- ☐ usuário pode editar tarefas;
- ☐ usuário pode excluir tarefas (com confirmação);
- ☐ usuário pode iniciar timer;
- ☐ usuário pode pausar timer;
- ☐ usuário pode parar timer;
- ☐ somente uma tarefa pode estar ativa;
- ☐ tempo é persistido corretamente;
- ☐ timer sobrevive ao fechamento/reabertura;
- ☐ tarefas podem possuir múltiplas sessões;
- ☐ tags podem ser criadas;
- ☐ tags podem ser editadas;
- ☐ tags podem ser excluídas;
- ☐ tags possuem nome, descrição e cor;
- ☐ painel lateral direito funciona;
- ☐ sidebar esquerda funciona;
- ☐ Settings funciona;
- ☐ Dark Mode funciona;
- ☐ Light Mode funciona;
- ☐ limpeza de histórico possui confirmação;
- ☐ aplicação funciona offline;

- [] não existe dependência de servidor;
 - [] não existe funcionalidade futura implementada prematuramente;
 - [] testes principais foram executados;
 - [] aplicação não apresenta erros críticos.
-

62. BACKLOG FUTURO

As funcionalidades abaixo NÃO fazem parte do MVP.

Podem ser consideradas posteriormente:

V1.1

- Dashboard;
- estatísticas dos últimos 7 dias;
- tempo por tag;
- tempo por tarefa;
- filtros;
- busca;
- status de tarefa (concluída/arquivada) com UI dedicada;
- edição granular de múltiplas sessões (TimeEntry) por tarefa.

V1.2

- Pomodoro;
- notificações;
- exportação;
- relatórios.

V2+

- projetos;
- subtarefas;
- metas;

- backup;
- importação/exportação avançada;
- ícones nas tags;
- recursos adicionais de produtividade.

Nenhuma dessas funcionalidades deve ser implementada durante o MVP sem aprovação explícita.

63. REGRAS ESPECÍFICAS PARA O CLAUDE CODE

Estas regras possuem prioridade durante o desenvolvimento.

Regra 1 — Uma fase por vez

Nunca executar duas fases simultaneamente.

Regra 2 — Não avançar automaticamente

Depois de concluir uma fase, parar.

Não iniciar a próxima fase sem a mensagem explícita do usuário autorizando.

Exemplos de autorização:

Pode continuar.

Vamos para a próxima fase.

Aprovado.

Regra 3 — Não inventar requisitos

Se algo não estiver definido:

- identificar a dúvida;
- explicar as opções;
- recomendar uma solução;
- aguardar decisão quando a escolha afetar arquitetura ou UX.

Não inventar funcionalidades.

Regra 4 — Não refatorar sem necessidade

Não realizar grandes refatorações durante uma fase sem necessidade.

Se uma mudança arquitetural for necessária:

1. explicar;
2. justificar;
3. apresentar impacto;
4. aguardar aprovação quando relevante.

Regra 5 — Não adicionar dependências desnecessárias

Antes de adicionar uma biblioteca:

- verificar se a funcionalidade já pode ser resolvida com .NET/WPF;
- avaliar impacto;
- explicar a necessidade.

Regra 6 — Não esconder erros

Se algo falhar:

- informar;
- explicar causa provável;
- corrigir;
- testar novamente.

Não simplesmente contornar o problema silenciosamente.

Regra 7 — Testar cada etapa

Toda fase deve terminar com validação.

Regra 8 — Preservar funcionalidades existentes

Uma alteração nova não deve quebrar funcionalidades já aprovadas.

Regra 9 — Código limpo

Priorizar:

- nomes claros;
- classes pequenas;
- métodos objetivos;

- baixo acoplamento;
- alta coesão.

Regra 10 — Não overengineer

Não criar:

- abstrações desnecessárias;
- factories sem necessidade;
- múltiplas camadas sem função;
- padrões complexos apenas para parecer arquiteturalmente sofisticado.

A arquitetura deve ser suficientemente robusta para o projeto, não maior que ele.

64. FORMATO OBRIGATÓRIO DE RESPOSTA DO CLAUDE AO FINAL DE CADA FASE

Ao finalizar uma fase, Claude Code deverá responder seguindo este formato:

FASE X — CONCLUÍDA

Implementado:

- item
- item
- item

Arquivos criados:

- arquivo
- arquivo

Arquivos modificados:

- arquivo
- arquivo

Testes realizados:

- teste

- teste

Resultado:

- PASSOU / FALHOU

Problemas encontrados:

- nenhum

ou

- descrição

Observações:

- descrição

PRÓXIMA FASE:

FASE X+1

Aguardando aprovação para continuar.

Claude NÃO deve executar a próxima fase após essa mensagem.

65. CRITÉRIO DE DECISÃO TÉCNICA

Quando houver mais de uma solução possível, priorizar nesta ordem:

1. Simplicidade

2. Confiabilidade

3. Manutenibilidade

4. Compatibilidade com Windows

5. Testabilidade

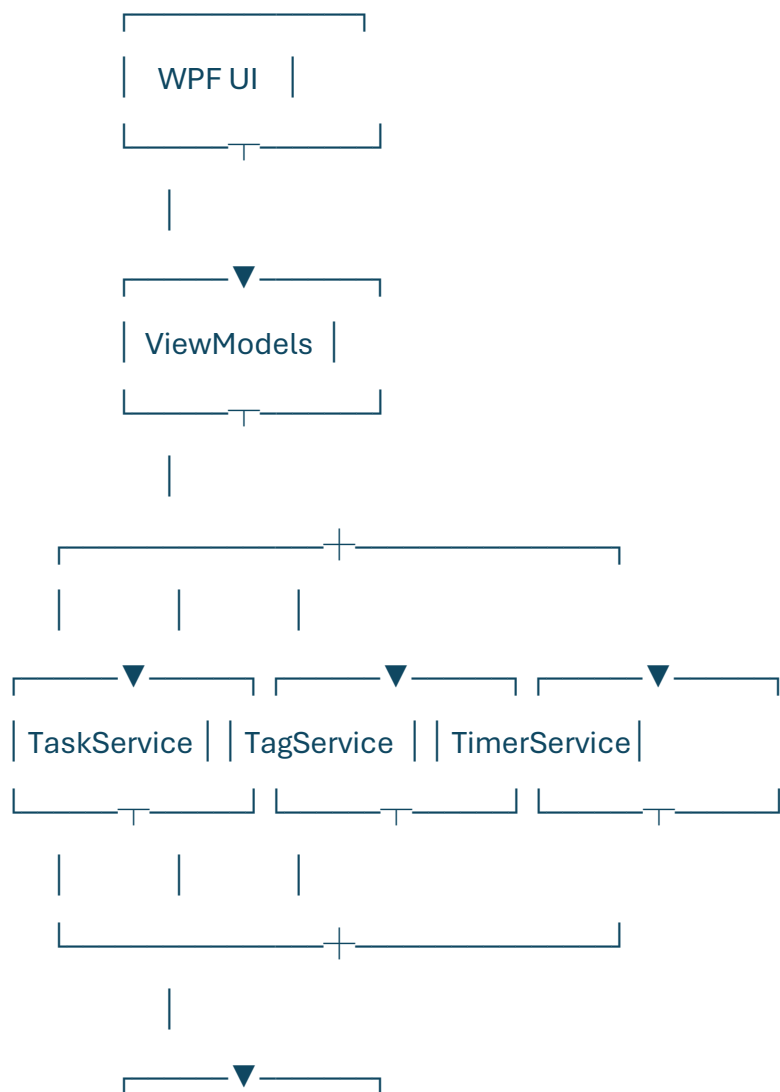
6. Extensibilidade

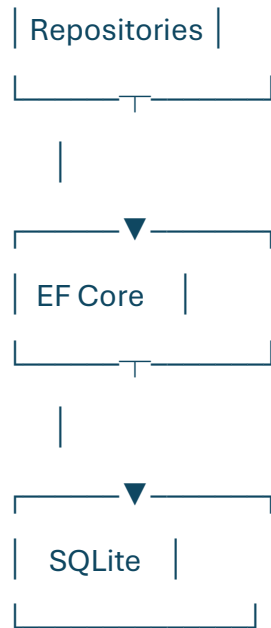
7. Performance

Não priorizar "tecnologia da moda" quando ela não trazer benefício concreto para este projeto.

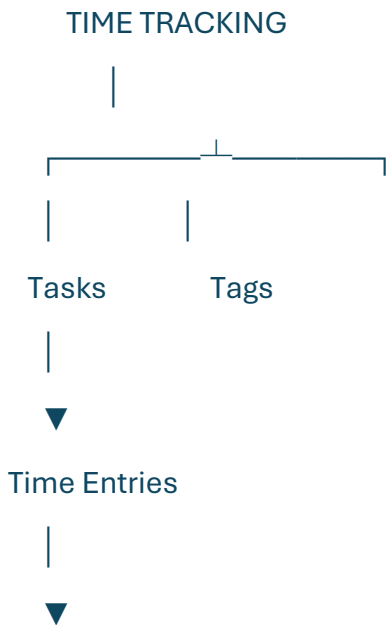
66. VISÃO FINAL DA ARQUITETURA

O resultado esperado do MVP é:





O núcleo do produto deverá permanecer:



Tudo que for desenvolvido futuramente deverá se apoiar nesse núcleo, e não modificar desnecessariamente sua lógica.

67. PRIMEIRO COMANDO PARA INICIAR O DESENVOLVIMENTO

Ao receber este documento, Claude Code deve iniciar **somente pela FASE 0**.

Não criar a aplicação inteira.

Não implementar a FASE 1.

Não criar todas as Views.

Não criar o banco ainda.

Primeiro:

1. analisar a especificação;
2. validar a arquitetura;
3. identificar inconsistências;
4. apresentar decisões técnicas necessárias;
5. apresentar o plano da FASE 0;
6. aguardar aprovação.

A implementação só começa após a aprovação explícita da FASE 0.

Fim da especificação.